

AsymptoticAnalysis

After the Mathics merge

An incremental audit of correctness boundaries,
native Wolfram capabilities, interpreter compatibility,
and concrete implementation opportunities

Prepared for Vladimir Reshetnikov
Repository VladimirReshetnikov/Asymptotic
Revision 7d1bc832895cc90a9b2a978b7b7684acab908bd2
Audit date September 9, 2026, America/Los_Angeles

What is new here. Three reproduced defects in the new portable test runner; a source-level early-termination issue in a Mathics adapter; precise interpretation of six current-revision CI artifacts; and two implemented mathematical prototypes: formal Newton inversion of the normalized Gamma equation and stable numerical evaluation of the real W_{-1} branch.

What this is not. A repetition of the existing 27 reviews, a claim of complete Mathics compatibility, or a report of a newly reproduced wrong public asymptotic formula. Executed kernel observations, upstream CI evidence, source inspection, exact independent algebra, and ordinary numerical experiments are kept distinct.

Abstract

This report reviews the Asymptotic repository at the Mathics-compatibility merge 7d1bc83. The package’s distinctive contribution is not merely another Taylor expander: it combines sparse real power-log calculus, selected non-power scales, inverse constructions, explicit branch and remainder information, and a separate contract for native Wolfram results. Its Mathics support is now substantial implementation work rather than a hypothetical port. Six retrieved CI artifacts contain 69 successful completed records, covering 65 distinct test IDs, but only two of those suites completed and the overall workflow was cancelled.

The audit adds reproducible runner defects concerning timeout admission, dependency coverage through the official `init.m` loader, and interrupted in-flight tests. A focused candidate patch passes six independent harness tests using fake kernels. A separate exact Gamma-inverse prototype agrees between triangular and precision-doubling Newton algorithms through order eight; a stable W_{-1} prototype passes branch-point and extreme-target numerical checks. Those prototypes do not constitute accepted Wolfram/Mathics package patches, analytic error certificates, or native performance measurements. Existing review items are used as a deduplication baseline, with new implementation detail explicitly identified when a proposal sharpens an older item.

Contents

1	Assessment and scope	4
1.1	Overall assessment	4
1.2	Revision, acquisition, and coverage	4
1.3	Evidence classes	4
2	What the package actually adds	5
2.1	A sparse real germ rather than just a polynomial	5
2.2	Four interacting layers	6
2.3	Formal identity, asymptotic theorem, and pointwise certificate	6
3	Comparison with native Wolfram Language	7
3.1	Compare contracts, not just function names	7
3.2	A concrete order-convention check	8
3.3	Native delegation is not analytic certification	9
3.4	Using native tools as references without making them an oracle	9
4	Mathics compatibility at the pinned revision	9
4.1	The port is real, but compatibility is not complete	9
4.2	Evaluator semantics: where apparently simple ports fail	10
4.3	The proof boundary is intentionally conservative	10
4.4	Special-function bridges have bounded mathematical domains	10

4.5	Current-revision CI: what actually completed	11
4.6	Timing and preservation evidence require separate interpretations	12
4.7	Practical UX implications	12
5	N1: unsafe timeout admission in the portable runner	12
5.1	Mechanism and witness	12
5.2	Repair and policy boundary	13
5.3	Novelty and severity	13
6	N2: source provenance misses the official init loader	13
6.1	The dependency condition is too narrow	13
6.2	Why the current CI counts remain usable	14
6.3	Candidate repair	14
6.4	Novelty and severity	15
7	N3: interrupted in-flight cases disappear from the receipt	15
7.1	The current checkpoint records completions, not attempts	15
7.2	Write-ahead case state	15
7.3	Remaining acceptance work	16
8	N4 and D1: two bounded adapter observations	16
8.1	N4: FirstPosition enumerates every match	16
8.2	D1: the Limit adapter's Automatic default is one-sided	17
9	P1: a concrete Gamma-specific formal Newton engine	17
9.1	Why this is additional work rather than "add Newton"	17
9.2	Normalize around the exact dominant inverse	18
9.3	The coefficient ring and the update	18
9.4	Low-order coefficients and independent checks	19
9.5	Original-function numerical checks	20
9.6	Integration and performance acceptance	20
10	P2: a stable numerical route for the real Lambert branch W_{-1}	20
10.1	Scope of the proposal	20
10.2	Avoid representing the tiny argument at all	21
10.3	A global real bracket	21
10.4	Stable phase evaluation and bounded bisection	21

10.5 Numerical evidence	22
10.6 A narrow path into Mathics	22
11 Additional development consequences of the Mathics architecture	23
11.1 Treat late definition rewrites as versioned transformations	23
11.2 Measure startup separately from mathematical work	23
11.3 Keep internal proof allowances distinct from user deadlines	23
12 Incremental roadmap and novelty ledger	24
12.1 Recommended work order	24
12.2 What was deliberately not re-reported	24
12.3 Acceptance matrix for the supplied artifacts	25
13 Reproduction and bundle contents	26
13.1 Run the independent checks	26
13.2 Native and Mathics characterizations	26
13.3 Rebuild the article	27
14 Conclusion	27
A Source coverage and preserved artifact provenance	27
A.1 Repository files inspected	27
A.2 CI artifact identities	28
References	29

1 Assessment and scope

1.1 Overall assessment

The repository is best understood as a specialized asymptotic calculus with a native delegation layer, not as an already complete replacement for the Wolfram asymptotics ecosystem. Its most useful design choice is that a finite expression, an asymptotic error statement, a branch condition, and a numerical certificate are not treated as interchangeable objects. The native wrapper deliberately preserves a weaker, formal result contract instead of inventing a package remainder for whatever a built-in returns. The current Mathics implementation similarly attempts bounded exact reasoning and conservative refusal rather than replacing unproved symbolic facts with floating-point guesses. These are good foundations to preserve.[1, 2, 7, 11]

The strongest new issues established here are in *validation machinery*, not in an observed public mathematical answer. They matter because an ambitious compatibility effort depends on being able to say exactly which source was tested, which case ran, and why a run stopped. None of the runner findings demonstrates a false successful assertion or an unsound interval certificate. Their severity is medium for evidence integrity or operational reliability, not critical mathematical corruption.

There is also a concrete source-level performance problem in the compatibility implementation of `FirstPosition`: it obtains all matching positions and only then selects one. A lower-priority `Limit` default discrepancy is documented separately because inspected consumers already supply explicit directions. These qualifications are important: a private adapter mismatch is not automatically a reproducible wrong public expansion.

1.2 Revision, acquisition, and coverage

All repository claims in this report refer to 7d1bc832895cc90a9b2a978b7b7684acab908bd2. Its commit timestamp is September 10, 2026, 02:10:22 UTC, which is September 9, 19:10:22 PDT. The date on the title page is therefore consistent with the repository timestamp. The pinned standalone file loaded in the successful native smoke test was 664,603 bytes. The package identifies itself as version 1.8.0.[1, 2]

Sources were read through the GitHub connector, including the root and kernel documentation, the complete consolidated review crosswalk and wave-3 intake, native dispatch, Gamma and Lambert inverse construction, Fourier coefficient machinery, the portable runner, CI configuration, and the Mathics compatibility modules. This was not a full local checkout or a claim that every line of every mathematical engine was independently re-proved. The detailed source-coverage appendix lists the reviewed areas.

Deduplication used the central register covering 167 ledger entries across 27 reports, the wave-3 intake covering its 44 entries, and selected original report material. The count is a count of report entries, not 167 distinct unresolved defects. It would be inaccurate to claim that all 27 complete articles were individually reread for this audit. The register's explicit crosswalk is the primary exclusion mechanism.[4, 5, 6]

1.3 Evidence classes

Class	Meaning in this report
Native observation	A successful call to the Wolfram 15.0.0 Linux evaluator during this audit. One current-package smoke calculation and several native <code>Limit</code> controls succeeded.
Retrieved CI	Original JSON artifacts from the repository's pinned GitHub Actions run, not executions performed locally by this audit.
Executed harness	The byte-identical upstream Python runner and a candidate patch exercised with synthetic sources and fake kernel processes. This isolates runner behavior.
Exact independent model	New Python/SymPy code performing exact finite formal algebra. It does not execute the Wolfram package or certify its integration.
Numerical experiment	Arbitrary-precision mpmath comparisons and residuals, without directed rounding or interval certification.
Source analysis	A claim supported by a specific definition and its control flow, with runtime or public reachability limits stated.
Proposal	A design or patch requiring integration acceptance; passing a prototype is not silently promoted to package acceptance.

The local environment did not provide a working Mathics installation. Installation and repository-download attempts failed at the network layer. The Wolfram connector initially failed, subsequently accepted a few calls, and later returned network/HTTP 502 errors. Those service failures are not defects attributed to the repository. They constrain the execution evidence: no full upstream suite, native Gamma benchmark, or new Mathics adapter regression was run by this audit.

2 What the package actually adds

2.1 A sparse real germ rather than just a polynomial

A useful schematic form of the package's power-log object is

$$S(u) = \sum_{\alpha \in A} u^\alpha P_\alpha(\log u) + O\left(u^\beta (1 + |\log u|)^k\right), \quad u \rightarrow 0^+, \quad (1)$$

with a finite retained support A , exact real weights, and polynomial logarithmic coefficients. The object also retains the coordinate relating u to the user's variable, branch information, assumptions, and, for supported operations, construction/refinement data. The precise stored representation depends on the family; equation (1) is a mathematical summary, not a claim that every object uses one universal schema.[2, 3, 7]

The positive local coordinate is substantive. At a finite endpoint one may have $u = x - x_0$ or $u = x_0 - x$; at infinity one may use a reciprocal. A noninteger power of u can then be real and single-valued without pretending that an arbitrary complex power has the same branch. Once coordinates are explicit, a cutoff, an observable power, and the side of approach can be interpreted consistently.

For example, put $\alpha = \sqrt{2}$ and consider $f(x) = x + x^\alpha$ for $x > 0$ near zero. Set $x = yv$ and $t = y^{\alpha-1}$. The inverse equation becomes

$$v + tv^\alpha = 1.$$

At $(v, t) = (1, 0)$ the derivative with respect to v is one, so the local real solution has an ordinary analytic expansion in t . Substituting its first coefficients gives

$$f^{-1}(y) = y - y^\alpha + \alpha y^{2\alpha-1} - \frac{\alpha(3\alpha-1)}{2} y^{3\alpha-2} + \dots$$

This is an illustrative mathematical derivation, not an additional executed package probe. Its exponent set is perfectly natural for a sparse weighted algebra but cannot be represented as a faithful single rational SeriesData lattice with coefficients independent of the expansion variable.

Logarithmic coefficients are another reason to preserve blocks. Formal substitution into $f(x) = x + x^2 \log x$ yields

$$f^{-1}(y) = y - y^2 \log y + y^3 (2 \log^2 y + \log y) + \dots$$

The y^3 contribution is a complete polynomial block, not two unrelated orders. A term goal based on complete nonzero blocks has different semantics from counting printed summands. This is not a newly discovered API flaw; it is a distinction needed to use and compare the implemented APIs correctly.[2, 4]

2.2 Four interacting layers

The source organization suggests four practical layers. The first handles held requests, callables, option resolution, coordinates, and source admission. The second contains specialized coefficient and scale engines: power-log, logarithmic, Fourier, flat-sector, Lambert, Gamma/Barnes, and other selected special-function paths. The third handles GeneralizedSeries operations, remainders, derived objects, and refinement. The fourth provides native delegation, numerical/certificate utilities, distribution, and interpreter adapters.[3, 7, 11]

These layers should not be confused with an abstract architecture proposed for a future rewrite. They summarize already implemented responsibilities. Existing binary powering, sorted sparse-product pruning, grouped Lagrange machinery, generic Newton work, and retained coefficient state are also already present. Recommending that the project simply “add Newton” or “use sparse multiplication” would recycle earlier reviews and miss the actual implementation.[4]

The package’s inverse families are particularly valuable when a dominant core should remain exact. For a Gamma inverse, keeping a Lambert- W expression for the dominant scale avoids immediately expanding one slow variable inside another. Fourier coefficients in $\log u$ retain oscillatory structure without forcing it into polynomial-log coefficients. Flat sectors retain exponential smallness that disappears under every fixed algebraic cutoff. These are different coefficient/scale contracts, not merely more special-function names.[8, 9, 10]

2.3 Formal identity, asymptotic theorem, and pointwise certificate

Suppose an approximate inverse is g_N and the computed residual is $f(g_N(y)) - y$. Exact cancellation of finitely many coefficients proves a finite algebraic identity in the adopted model. To turn that into an asymptotic error statement one also needs a valid source remainder and inverse stability. To get a pointwise bound one needs quantitative information, for example a verified lower bound on $|f'|$ on a suitable interval. A tiny numerical residual alone establishes none of those missing hypotheses.

The repository exposes operations corresponding to these different tasks and explicitly distinguishes native formal output from a package analytic remainder. Its Gamma construction labels the forward

Stirling model as finite Poincaré information, not a convergent power series or pointwise inverse certificate. The Mathics certificate adapter changes the representation of a final list assignment rather than substituting numerical agreement for interval logic.[7, 8, 19]

This distinction should govern performance work as well. A faster computation of the same finite coefficients does not, by itself, improve the remainder theorem; conversely, strengthening an error envelope should not require recomputing coefficients that have already been proved unchanged. The Gamma prototype below deliberately targets only the coefficient engine.

3 Comparison with native Wolfram Language

3.1 Compare contracts, not just function names

Native `Series` computes local series, including multivariate sequential expansions and familiar non-Taylor behavior. `SeriesData` stores a finite coefficient list with a rational exponent lattice and an order boundary. `InverseSeries` performs formal reversion of supported series. Those facilities are substantial; the package should not be credited with inventing fractional-power series or formal inversion.[30, 31, 32]

Native `Asymptotic` goes beyond Taylor expansion, and `AsymptoticSolve` supports implicit equations, systems, real-domain requests, complex centers, and scales inferred from the problem. Its order parameter is not promised to equal a polynomial degree. `DiscreteAsymptotic` is a distinct native capability for discrete sequences. The package's "Backend" values are `Automatic`, `"Package"`, `"Series"`, and `"Asymptotic"`; the broader aspiration to subsume native coverage is not an implemented discrete backend.[33, 34, 35, 7, 2]

Task	Native baseline	Package's practical distinction
Ordinary local expansion	<code>Series</code> ; <code>Asymptotic</code> when scale selection matters.	Real coordinate normalization and a structured remainder/branch object; native delegation remains available.
Formal reversion	<code>InverseSeries</code> ; implicit formulations via <code>AsymptoticSolve</code> .	Sparse weighted inverse coefficients, selected observable powers, explicit inverse-family metadata and residual operations.
Irrational power support	Expressions may contain irrational powers, but a single ordinary <code>SeriesData</code> lattice is not a general sparse real-weight store.	Exact independent real weights are first-class within admitted custom scales.
Logarithmic blocks	Native asymptotic output can involve logarithms.	Complete polynomial-log blocks participate in the package's cutoff and error calculus.
Special inverse cores	Native symbolic special functions and equation solvers provide ingredients.	Selected Lambert, Gamma/Barnes and other inverse constructions retain a chosen core and its scale-specific remainder.
Operations on a constructed germ	Built-in series arithmetic and composition operate on native representations.	Domain, uncertainty and construction state accompany supported arithmetic, composition and refinement.

Task	Native baseline	Package's practical distinction
Oscillatory or flat scales	Native coverage depends on the expression and asymptotic problem.	Explicit finite Fourier-log and selected exponentially small sector machinery. This is not unrestricted transseries closure.
Complex or multivariate problems	Native tools have broader documented scope.	Custom analytic paths remain constrained; returning a native wrapper does not make the custom real calculus complex-complete.
Discrete asymptotics	Dedicated <code>DiscreteAsymptotic</code> .	Not one of the current selectable native backends.
Numerical guarantees	Numerical solving and symbolic series are separate native tasks.	Selected inverse certification facilities are separate from <code>Normal</code> and raw numerical substitution.

The table is a capability and representation comparison, not a benchmark and not a declaration that native tools fail every problem assigned to a custom family. A meaningful comparative test must match the branch, direction, observable, achieved cutoff, coefficient granularity, and kind of error evidence. Otherwise a shorter answer can simply be a less demanding answer.

3.2 A concrete order-convention check

The following custom calculation was actually executed against the pinned standalone source in Wolfram 15.0.0 on Linux:

```
s = AsymptoticAnalysis'AsymptoticExpansion[
  Exp[x], {x, 0, 4}, "Backend" -> "Package"];
{Normal[s], s["Remainder"]}
(* {1 + x + x^2/2 + x^3/6, PowerLogRemainder[x,4,0]} *)
```

The custom cutoff is exclusive. To compare the finite polynomial with ordinary native Taylor output, the corresponding native degree is three:

```
Normal[Series[Exp[x], {x, 0, 3}]]
```

The second line states the matching mathematical/native specification; it is not recorded as another package test. Copying the numeral four into both calls would compare different retained information. For irrational cutoffs and non-power cores there may be no analogous single integer translation.

The successful native load returned `Null`, and `Names["AsymptoticAnalysis'"]` had length 38. These are smoke observations, not API-completeness or behavioral-parity proofs. The manually transcribed record is preserved in `results/native-observations.json`; the reusable reconstruction is `code/native_smoke.wls`.

3.3 Native delegation is not analytic certification

In `NativeCompatibility.wl`, `nativeSeriesQ` distinguishes objects whose kind is "Native", and `requireAnalyticSeries` refuses operations that demand a custom analytic remainder. The native result preserves its raw payload, request and evaluation status. An unresolved built-in and a computed native result are different outcomes; neither automatically becomes a package theorem.[7]

This separation makes the facade useful even before the custom engine covers a given expression. It also means that "the facade returned an object" is a poor success criterion. Tests should inspect result kind and raw native output before assuming a custom operation such as remainder-aware refinement will apply.

Existing wave-3 reports already analyze omitted backend defaults, equivalent option keys, overbroad callable protection, eager native `Normal`, and held syntax in outcome classification. Those findings are not republished here. Their relevance to this comparison is only that the native facade's current syntax surface is broader than its settled request-resolution contract.[5]

3.4 Using native tools as references without making them an oracle

For regular small polynomial inverses, `InverseSeries` is an excellent independent coefficient comparison. For implicit branches, `AsymptoticSolve` can provide a second construction with explicitly aligned real-domain and direction conditions. For special functions, a native result is valuable evidence but should be checked against a defining relation or a separate coefficient formula rather than being declared true merely because it came from the proprietary kernel.[32, 34]

The added Gamma tests use this principle in a different way: an independent exact truncated-ring engine is compared with a separate symbolic defining-expression calculation at low order, followed by numerical checks against the original *logarithm of Gamma*, not just its finite Stirling polynomial. Agreement between two algorithms sharing a phase evaluator is useful, but that shared dependency is explicitly accounted for rather than presented as complete independence.

4 Mathics compatibility at the pinned revision

4.1 The port is real, but compatibility is not complete

The current documentation states complete Mathics compatibility as a goal, not an achievement. It targets Mathics3 10.0.1 with scanner 10.0.1, SymPy 1.14.0 and Python 3.11. Selected symbolic, arithmetic, inverse, special-function and certificate examples already have focused evidence. That is a substantially stronger position than "the package might load in Mathics," but a substantially weaker position than "it behaves identically to Wolfram Language." [11, 38]

The package isolates compatibility definitions in its own context and loads them only on Mathics. Early definitions affect how later package code resolves selected names; late adapters transform selected private definitions. The public context path is restored after loading. Some missing `System`` names are interned so user input and package patterns refer to the same symbols, but interning an inert `Asymptotic`, `Reduce` or special-function name does not implement that function.[12, 11]

This isolation avoids global redefinitions of `System`Map`, `System`Simplify` and `System`ProductLog`. It limits collateral effects on other packages and gives the Wolfram path a plausible preservation boundary. It does not eliminate the need to test the transformed definitions themselves.

4.2 Evaluator semantics: where apparently simple ports fail

The Module/Return adapter creates a fresh dynamic catch tag around each native module. This handles the package's explicit module-directed returns through loops and nested helper calls. A syntactic substitution of `Return[value,Module]` with an ordinary Mathics return would not establish the same control flow. The portable foundation suite contains direct tests of this behavior.[12, 11]

Association support similarly concerns evaluation timing, not only key/value representation. Lookup holds its default until a missing key actually requires it; the list-of-keys form must preserve order. Refinement adapters evaluate computed rule lists before constructing associations. Held extraction for named function parameters prevents a formal parameter's ownvalue from being evaluated while the package is merely inspecting callable syntax.[12, 16, 17]

Other fixes address interpreter-specific hazards without changing the intended mathematics. Finite derivative sums are expressed through Table and Total so indices are bound before derivative evaluation. The certificate adapter replaces a last-element negative index assignment with the equivalent positive index. The empty-list mapping adapter avoids a Mathics cache-state problem and rewrites only package-private references to Map. Formatting/arithmetic upvalues have a separate installation path because Mathics rejects the original tagging pattern.[18, 19, 20, 21]

These details explain why a feature inventory of built-in names alone is not an adequate compatibility assessment. Even an implemented Map, Sum, Module or Association can differ at an evaluation boundary crucial to this package.

4.3 The proof boundary is intentionally conservative

The assumption layer establishes selected exact facts from explicit assumptions: finite realness, signs, products, powers and bounded polynomial/affine reasoning. It does not claim to implement quantifier elimination. An unresolved sign is not silently replaced by positivity, and a disjunction is not treated as one of its disjuncts. The simplification wrapper also avoids feeding unresolved ordered relations with potentially nonreal operands into native simplification.[13, 14, 15]

That last restriction is worth emphasizing. Cancellation in $a + u > a$ is harmless only when the order relation's operands are in the intended ordered domain. Algebraically cancelling a shared complex offset and obtaining $u > 0$ does not prove the original real inequality. The adapter explicitly protects that boundary. Current foundation tests include corresponding nonreal-operand and disjunction controls; this audit is recognizing an existing safeguard, not reporting a missing one.[11]

A refusal caused by an unavailable proof is a compatibility limitation, but it may be the correct behavior under the mathematical contract. Compatibility work must distinguish increasing the set of provable inputs from weakening the criteria required to accept an input. The latter can make a demo succeed while invalidating the package's reason for retaining assumptions and branches.

4.4 Special-function bridges have bounded mathematical domains

The defining-series bridge admits selected hypergeometric functions and PolyLog at a vanishing monomial argument, with an outer constant multiplier, exact arity checks and explicit order terms. Hypergeometric denominator parameters are restricted to proved-positive values, and the convergent defining-series condition is bounded. This avoids silently applying an origin formula to an unsupported continuation or letting a surrounding pole amplify an omitted coefficient. It is not a general replacement

for native special-function asymptotics.[22]

For a growing positive argument, the Mathics LogGamma bridge supplies a finite Stirling model to the existing jet algebra. It enforces a finite working order and adds a Poincaré remainder rather than marking the result exact. Principal Lambert expressions are normalized to one-argument ProductLog at selected package construction sites; nonprincipal numerical Lambert evaluation remains explicitly unsupported in the compatibility notes.[23, 24, 11]

The current negative-Lambert family CI success below is a *symbolic family* test. It does not contradict the documented numerical W_{-1} limitation. A test that successfully constructs a branch-tagged expression and a test that correctly specializes that expression numerically are different acceptance obligations.

4.5 Current-revision CI: what actually completed

The GitHub Actions run 34428390335 corresponds to the pinned commit. Its overall conclusion is cancelled. The modular foundation and contracts jobs succeeded; the remaining jobs were cancelled, including some that did not progress far enough to produce a complete test receipt. Six downloadable artifacts were retrieved and their JSON records preserved.[29, 28]

Entry / suite	Selected	Records	Passes	No record	Complete?
Modular / foundation	20	20	20	0	Yes
Modular / contracts	21	21	21	0	Yes
Modular / calculus	24	13	13	11	No
Modular / special	13	8	8	5	No
Modular / operations	12	3	3	9	No
Standalone / foundation	20	4	4	16	No
Retrieved receipts	110	69	69	41	Two suites

Table 3: Counts calculated from the preserved original receipts. “No record” means the receipt has no completed result, not proof that a case never started. No assertion failures appear among the 69 completed records.

The 69 successful records cover 65 distinct test IDs because four standalone loading cases duplicate modular foundation IDs. The five modular suites select 90 IDs in total; the configured two-entry-point matrix is therefore larger than the six available receipts. Neither 69/69 nor 65/65 is an honest headline pass rate for that full matrix. The appropriate statement is that all *completed recorded* checks passed, with substantial uncompleted coverage remaining.

Every preserved receipt reports unchanged tested sources. The runtime identifies Mathics3 10.0.1 on CPython 3.11.16/Linux, with SymPy 1.14.0 and mpmath 1.3.0. Each case starts a fresh process, uses a 300-second timeout including startup, and configures the suite’s Mathics iteration limit to one million. These facts come from the artifacts rather than being inferred from the advertised dependency file. The script code/summarize_ci.py regenerates Table 3’s data.

The completed calculus records include polynomial/exponential/logarithmic forward expansions, irrational exponents, a finite approach from below, decaying exponentials, quadratic and ramified inversion, and finite/infinite source points. Special records include Gamma and Barnes forward corrections, Bessel/Erf/PolyLog origin cases, a Zeta Dirichlet example, and selected inverse families. Contract records include named and slot callables, branch/domain refusals, explicit native-series delegation, exact rational certification, and a fixed-center accuracy-floor case. These are concrete examples, not proof of every parameter range of the corresponding family.

4.6 Timing and preservation evidence require separate interpretations

The preserved timing fields include interpreter startup and package loading. Modular loading examples generally took about 8–10 seconds on that CI run, while one Barnes forward example took about 98 seconds. Those numbers identify expensive end-to-end cases for further measurement; they are not kernel-only timings, stable medians, or a native-versus-Mathics speed ratio. The jobs can also run concurrently. Claiming a speedup from comparing unrelated receipt entries would be unjustified.

The repository separately documents a Wolfram preservation run with 1,452 passes and the same 12 existing failures across 79 suites, followed by more recent definition comparisons against later controls. The documentation explicitly does *not* relabel that earlier full-suite result as a full run of every later merge. Definition equality across many attributes and value tables is useful preservation evidence, but it is not a fresh mathematical suite result for this pinned revision.[11]

The new native smoke test in this audit is narrower still. It shows that one exact current standalone file loads and performs an ordinary custom expansion under Wolfram 15.0.0/Linux. It cannot establish all Wolfram versions, notebook front ends, surrounding program interactions, or Mathics behavior.

4.7 Practical UX implications

The current setup instructions appropriately require an isolated supported Python environment, streaming package loading before parsing first use, and explicit session iteration-budget adjustment for Mathics. A single command string that parses an unknown public symbol before Get can bind it in the wrong context; this is not fixed by the package later exporting a symbol with the same short name. Windows also needs the documented UTF-8 and command-line startup handling.[11]

For result inspection, `Normal[s]`, `s["Remainder"]` and `InputForm[s]` are more reliable compatibility probes than an attractive notebook box. For testing, a clear refusal, unresolved native call, formal native object, and custom analytic object should be counted separately. Calling all four “compatible” would obscure the exact contract a user can rely on.

5 N1: unsafe timeout admission in the portable runner

Evidence: Reproduced against the byte-identical upstream Python file using fake kernels; the candidate fix was exercised on the same fixtures. No Wolfram or Mathics execution is involved.

5.1 Mechanism and witness

`validation/run_mathics_tests.py` accepts `-timeout` through `argparse` with `type=float` and checks only whether it is nonpositive. This admits `nan`, `inf`, and impractically large finite values. It writes the initial JSON report before handing the timeout to the subprocess communication machinery.[26]

```
# Existing admission condition
if args.timeout <= 0:
    parser.error("--timeout must be positive")
```

A NaN does not satisfy the comparison; positive infinity and `1e300` do not either. In the reproduced

Linux/Python 3.13 environment, these values reach lower-level timeout handling and produce `ValueError` or `OverflowError`, rather than a clean command-line diagnostic. A report file has already been created. For NaN/infinity its timeout field also uses Python's nonstandard JSON spellings, so a strict downstream JSON consumer has a separate reason to reject it.

The test harness creates a synthetic two-case suite and a fake executable that emits the runner's expected protocol. The fake executable sleeps briefly before reporting; this forces the runner through a real subprocess wait rather than merely testing a comparison expression. Ordinary finite-timeout controls complete successfully with both original and candidate runners. Invalid cases exit nonzero on the original, but the failure is a traceback after report creation, not a deliberate admission refusal.

5.2 Repair and policy boundary

The candidate uses

```
if not math.isfinite(args.timeout) or not 0 < args.timeout <= 86400:
    parser.error(
        "--timeout must be finite and positive, "
        "and at most 86400 seconds")
```

Finite positivity repairs NaN/infinity. The upper bound repairs huge finite conversion failures and is an explicit *runner policy*, not a theorem that a kernel cannot legitimately run longer than a day. The project may choose another defensible cap, but should not pass arbitrary floating-point magnitudes into platform wait primitives and expect uniform behavior.

The candidate refuses all three tested invalid inputs before starting a kernel or creating a report. Its six-test harness includes these subcases, ordinary controls, source-coverage tests and interruption tests. The shipped unified diff modifies only this runner; it is not automatically applied to the user's repository.

5.3 Novelty and severity

The prior V02 finding concerns the legacy aggregate test runner's discovery/report-success criteria. This is a different, recently added portable runner and a different failure mechanism. The general earlier demand for resource-budget discipline does not constitute this concrete timeout witness or its repair. Severity is operational reliability: this does not demonstrate a false-green test run or a wrong expansion.

6 N2: source provenance misses the official init loader

Evidence: Executed source-fingerprint comparison with the original function and a mutated synthetic modular tree. The official loader's contents were read directly.

6.1 The dependency condition is too narrow

The runner's `fingerprints` function always includes the entry file, suite and runner. It includes sibling kernel modules only when the entry has the exact basename `AsymptoticAnalysis.wl` and its

parent is named `Kernel`. The repository also supplies a real entry point, `src/Kernel/init.m`, whose complete code is

```
Get[FileNameJoin[
  {DirectoryName[$InputFileName], "AsymptoticAnalysis.wl"}]]];
```

The runner's `-source` option accepts an existing file, so using this official loader is not rejected. Yet the special sibling-coverage condition does not fire. A mutation to the loaded `AsymptoticAnalysis.wl` or its companions can therefore leave the fingerprint set unchanged.[26, 27]

The witness exercises the actual original `fingerprints` function, not a model of it: construct a temporary `src/Kernel` with `init.m` and a child package file, take fingerprints, modify the child, and compare again. The original result is unchanged; the candidate result changes. The synthetic child is never executed as Wolfram code.

6.2 Why the current CI counts remain usable

This finding does not invalidate the six downloaded receipts. Their commands use the canonical modular `AsymptoticAnalysis.wl` or the generated standalone entry. The omission arises when an accepted alternative loader is supplied. The distinction is important: a flaw in a supported path is not permission to discard unrelated successful evidence or to claim that the pinned CI tested unknown source.

It also does not imply that every custom loader can be dependency-traced automatically. Arbitrarily constructed `Get` paths are a larger problem. A runner should report what it covered instead of implying that a filename heuristic proves complete dependency capture.

6.3 Candidate repair

The patch recognizes both canonical modular entry filenames and includes sibling `.wl` and `.m` files. For other loaders it adds repeatable `-dependency-root` arguments whose source files join the recorded set. The report adds an explicit coverage label:

```
"CanonicalModular"
"ExplicitDependencyRoots"
"EntryFileOnly"
```

This makes the remaining limitation visible. An explicit root set is a declaration of coverage, not proof that a dynamically constructed external dependency cannot exist. Tests verify both the official loader case and an explicitly declared custom dependency root.

At integration, the intended source closure should be decided before a run begins. Changing the closure policy midway and comparing incomparable sets would create a different provenance error. A test fixture should also cover a relocated modular tree, since the original runner already documents that use case.

6.4 Novelty and severity

Prior reviews requested source identity and robust CI more generally. The new contribution is the accepted `init.m` route that defeats this particular runner's module inclusion, together with a demonstrated mutation and a tested repair. Severity is evidence integrity for that route. There is no allegation of malicious source replacement or of a current canonical CI false pass.

7 N3: interrupted in-flight cases disappear from the receipt

Evidence: A real POSIX SIGINT was delivered while an owned fake child process was running. Both original and candidate runner behavior were recorded. The Windows cleanup path was not exercised.

7.1 The current checkpoint records completions, not attempts

The runner writes an initial report and then writes another report after each completed case. `run_case` cleans up the process tree on an unexpected exception and re-raises. If interruption occurs during the first case, the last report still contains no result. Its fields can say

```
"RunComplete": false,  
"Executed": 0,  
"NotRun": 2,  
"Results": []
```

even though the first kernel was started and spent time evaluating. The fake kernel writes a startup marker before sleeping; the test waits for that marker before sending SIGINT. Thus the witness is not inferred from timing or from a printed "Running" line.[26]

The original design is not false-green: `RunComplete` is false and the process exits nonzero. The loss is diagnostic. A consumer cannot tell from the final preserved receipt whether no case started, one case hung, a particular input crashed the interpreter, or the runner was interrupted during startup.

The same schema issue limits the interpretation of the current cancelled CI artifacts. Their `NotRun` counts are best read as "no completed record." Without an active-case field, the receipt alone does not identify any in-flight case at cancellation. This report therefore does not invent names or outcomes for those missing records.

7.2 Write-ahead case state

The candidate writes a checkpoint before launching each case, containing its ID, group, start timestamp and `StartedWithoutResult` state. It adds separate `Started`, `Completed`, `NotStarted` and `ActiveTest` fields while retaining legacy fields for compatibility. Here `Started` denotes a runner case attempt entered; it is not a stronger assertion that `Popen` succeeded. A more elaborate state machine could distinguish `Launching` and `ChildStarted` later.

A caught `KeyboardInterrupt` terminates the owned process tree, preserves captured diagnostics, and returns a result with `Outcome` -> "Interrupted". The main loop records that outcome and stops. If an uncatchable termination occurs, the write-ahead checkpoint can at least retain the active case identity. This is not a promise that SIGKILL can execute final cleanup or write a final result.

For the same two-case witness, the patched report identifies one started attempt, one not-started case, and an explicit interrupted outcome. Existing full-success controls still pass. The test only targets subprocesses it launched; it does not kill processes by a broad interpreter name.

7.3 Remaining acceptance work

The candidate needs a Windows interruption/control-event test, not merely the existing `taskkill` branch read in source. It also needs forced termination while the active checkpoint exists, interruption between a result append and report replacement, and an export-failure test. Those are narrow acceptance obligations for this patch, not a restatement of a generic full-suite CI wishlist.

A useful contract is: every selected case is either not attempted, actively attempted with identity, or has a terminal record; a successful run has terminal successes for all selected cases; an incomplete run never loses an already journaled identity. Atomic report replacement already exists in the runner and should be preserved.

8 N4 and D1: two bounded adapter observations

8.1 N4: FirstPosition enumerates every match

Evidence: Current source inspection and algorithmic analysis. The additional Mathics runtime characterization is supplied but was not executed.

The compatibility helper in `MathicsCompatibility.wl` is

```
firstPosition[expr_, pattern_, default_HoldComplete, levels_, heads_] :=
  System`Module[{positions},
    positions = Position[expr, pattern, levels, Heads -> heads];
    If[positions === {}, ReleaseHold[default], First[positions]]];
```

For ordinary pure predicates this gives the first of the full search results, preserving the intended position ordering. The problem is that obtaining all positions is strictly more work than stopping at the first match. A flat list of n matching elements produces n position records even though the caller requests one. On nested input, both the traversal and the materialized result can be much larger than necessary.[12, 36]

There is also an effectful-predicate distinction to characterize. Consider a predicate that increments a counter and immediately returns true on the first list item. An early-stopping operation need not call it on later items; the full `Position` search does. Later predicate evaluation could emit messages, consume a budget or throw. This is a source-level compatibility concern, not a claim that one of the package's current public mathematical inputs has been shown to reach an effectful predicate.

The supplied code/`adapter_characterization.wl` records a counter-based test at level one with `Heads -> False`, plus a lazy-default control. It is deliberately marked unexecuted. The existing portable foundation test establishes a successful pattern-position example, but not this early-stopping/callback-count property.

The simplest proposed repair is a bounded `Position` call requesting at most one match, *provided that exact form is verified on the supported Mathics version*. Otherwise a traversal must preserve levels, heads, ordering and holding rules; an ad hoc flat-list loop is not a complete substitute for this helper. The lazy

default already implemented should remain lazy. No unvalidated global replacement is shipped as an accepted fix.

The nearest existing item is generic P08 profiling. The incremental contribution is the precise newly added helper, its avoidable all-match materialization, and a semantic acceptance test beyond equality of the returned position. A measured public-package speedup remains unestablished.

8.2 D1: the Limit adapter’s Automatic default is one-sided

Evidence: Adapter source inspection and successful native Wolfram 15 controls. No current public failure is alleged.

The Mathics wrapper declares `Direction -> Automatic` and maps `Automatic -> 1`, alongside the explicit string mappings. In Wolfram 15, the retrieved `Limit` documentation and actual `Options[Limit]` observation use `Direction -> Reals` by default. The successful controls were

```
Limit[Sign[x], x -> 0] (* Indeterminate *)
Limit[Sign[x], x -> 0, Direction -> Automatic] (* Indeterminate *)
Limit[Sign[x], x -> 0, Direction -> 1] (* -1 *)
Limit[Sign[x], x -> 0, Direction -> -1] (* 1 *)
```

Thus the adapter’s default mapping is not a general emulation of native `Limit`’s default. It is especially important not to import the `Automatic` convention of another function, such as `AsymptoticSolve`, and assume all asymptotics-related functions share it.[14, 37, 34]

However, the inspected package `Limit` consumers explicitly supply a direction, usually the positive local-coordinate approach. That reduces this finding to latent adapter contract debt. The repair should either make the adapter’s restricted explicit-direction requirement clear or implement and test a genuine two-sided default. Changing the global Mathics `Limit` is neither necessary nor desirable. This sharpens the older direction-policy discussion without re-reporting a public side-of-approach defect already covered elsewhere.

9 P1: a concrete Gamma-specific formal Newton engine

9.1 Why this is additional work rather than “add Newton”

The repository already has Newton machinery in its generic inverse implementation. The present proposal targets a different code path: `gammaInverseUnit` in `GammaInverse.wl`. That routine builds coefficients successively, repeatedly forming the normalized phase and asking for a series to the next order. Its surrounding frontier search can ask for progressively larger row sets. P08 already identified Gamma recurrence profiling as an opportunity; this section supplies a specific algebra, normalized equation, exact prototype and matched operation counts rather than restating that opportunity.[8, 4]

The proposed change is not a new Gamma asymptotic theorem. It is a way to compute the same finite coefficients while retaining the existing branch, transformed-source and Poincaré remainder obligations. It should be inserted behind the current family contract only after native differential acceptance.

9.2 Normalize around the exact dominant inverse

For the large positive inverse of $\Gamma(x) = Y$, write $T = \log Y$ and let z solve

$$z(\log z - 1) = T, \quad z = \frac{T}{W(T/e)}.$$

The branch is the large positive core. Put

$$t = z^{-1}, \quad q = (\log z)^{-1}, \quad c = \log(2\pi), \quad a = \frac{1 - cq}{2}, \quad x = z(1 + U).$$

The usual finite Stirling expansion for $\log \Gamma$ provides the source model; it is interpreted asymptotically with a finite remainder, not as a convergent Bernoulli series.[39]

After subtracting the core equation and dividing by $z \log z$, the formal equation is

$$\begin{aligned} \Phi(U, t; q, a) &= U + q((1 + U) \log(1 + U) - U) - at - \frac{qt}{2} \log(1 + U) \\ &+ q \sum_{k \geq 1} \frac{B_{2k}}{2k(2k-1)} t^{2k} (1 + U)^{1-2k} = 0. \end{aligned} \quad (2)$$

For a target coefficient through t^N , only $2k \leq N$ contributes; the displayed infinite summation is shorthand for compatible finite truncations. No convergence claim is needed for finite formal coefficient extraction.

Differentiating the finite formal expression gives

$$\Phi_U = 1 + q \log(1 + U) - \frac{qt}{2(1 + U)} - q \sum_{k \geq 1} \frac{B_{2k}}{2k} t^{2k} (1 + U)^{-2k}. \quad (3)$$

In particular, $\Phi_U(0, 0) = 1$. This is a unit for every formal value of a, q ; no assumption such as $q \neq 0$ is required to invert the Jacobian as a formal series.

9.3 The coefficient ring and the update

Treat a, q as independent indeterminates while computing. All coefficients lie in $\mathbb{Q}[a, q]$, and substitute $a = (1 - cq)/2$ only for the actual Gamma problem. This avoids repeatedly expanding logarithmic constants and makes exact polynomial normalization straightforward.

Proposition 9.1. *Suppose $U_m \in t\mathbb{Q}[a, q][[t]]$ satisfies $\Phi(U_m, t) = O(t^m)$. Then*

$$U_{2m} = U_m - \frac{\Phi(U_m, t)}{\Phi_U(U_m, t)} \pmod{t^{2m}}$$

satisfies $\Phi(U_{2m}, t) = O(t^{2m})$. The quotient is well-defined in the truncated formal ring.

Proof. The denominator has constant coefficient one, so it is a unit. The correction is in $t^m \mathbb{Q}[a, q][[t]]$. Taylor expansion in that correction cancels the first-order residual, while every term at least quadratic in the correction is divisible by t^{2m} . This is an identity in a finite quotient ring; it does not assume convergence of the Stirling series. $\square$

The included engine implements truncated addition, multiplication, unit inversion, logarithms and shifts. It computes $\log(1 + U)$ through its derivative $(1 + U)'/(1 + U)$ and formal integration. Odd and even inverse-unit powers are reused for the finite Bernoulli contributions. The precision schedule is $m \mapsto \min(2m, N + 1)$, where length $N + 1$ includes coefficients through degree N .

```
while precision < order + 1:
    target = min(2 * precision, order + 1)
    F, D = algebra.phase(u, target, with_derivative=True)
    correction = algebra.mul(F, algebra.inverse(D, target), target)
    u = algebra.add(u, algebra.scale(correction, -ONE, target), target)
    precision = target
```

The complete implementation is in `code/gamma_inverse_jet.py`. It has a prototype order cap of 24, exact arithmetic through SymPy's polynomial-ring representation, and explicit input validation. The correctness tests in this audit reached order eight; the larger cap is not a claim that every order up to 24 was benchmarked.

9.4 Low-order coefficients and independent checks

Writing $U = u_1t + u_2t^2 + u_3t^3 + \dots$, the first coefficients are

$$\begin{aligned} u_1 &= a, \\ u_2 &= -\frac{q}{12}(6a^2 - 6a + 1), \\ u_3 &= q \left[\left(\frac{1}{2} - a \right) u_2 + \frac{a(a-1)(2a-1)}{12} \right]. \end{aligned}$$

For each tested order $N \in \{0, 1, 2, 4, 6, 8\}$, the triangular and Newton implementations agree exactly and substitution into the finite phase leaves zero residual through the requested order. Because those two routes share some arithmetic code, a separate symbolic defining-expression calculation checks the coefficients through order four. The test file records both types of evidence.

Order N	Triangular products	Newton products	Triangular phases	Newton phases
1	1	3	1	1
2	16	22	2	2
4	100	101	4	3
6	297	168	6	3
8	663	479	8	4

Table 4: Actual prototype counts. A product is one nonzero coefficient multiplication at the jet level; it is not a scalar machine operation and does not include all polynomial-normalization cost.

Table 4 supports a specific conclusion: the prototype reduces repeated nonlinear phase evaluations and, at the larger tested orders, counted jet coefficient products. It does *not* support universal superiority. At low orders it performs more products. The recorded single-run Python timings are too small and too sparsely sampled to support a stable crossover estimate. The implementation uses elementary polynomial operations, not an asymptotically fast multiplication package. No $O(M(N))$ speed claim, native speedup, or Mathics speed ratio follows from these counts.

9.5 Original-function numerical checks

For cores $z = 20, 50, 100$, set the target exactly in logarithmic form as $T = z(\log z - 1)$ and numerically solve $\log \Gamma(x) = T$. Evaluate the finite formal coefficients at $q = 1/\log z$ and compare the constructed inverse. The observed absolute errors include The full JSON includes intermediate orders and residuals.

Core	Order 2	Order 4	Order 8
20	$6.2524 \cdot 10^{-6}$	$2.0627 \cdot 10^{-9}$	$7.7026 \cdot 10^{-15}$
50	$8.6640 \cdot 10^{-7}$	$5.3148 \cdot 10^{-11}$	$4.4085 \cdot 10^{-18}$
100	$1.7211 \cdot 10^{-7}$	$2.7815 \cdot 10^{-12}$	$1.4242 \cdot 10^{-20}$

Table 5: Independent mpmath experiments against the original log-Gamma equation. These are rounded measurements, not interval enclosures.

Decreasing error over this finite grid is useful evidence, not a theorem of monotone improvement for all truncation orders. Poincaré expansions can eventually diverge as more terms are added at a fixed argument.

9.6 Integration and performance acceptance

A production port should first compare exact coefficient polynomials with `gammaInverseUnit`, then compare final row construction for supported source translations, scales and observable powers. It must preserve zero-block filtering and first-omitted-block selection rather than assuming every formal coefficient is nonzero. The existing source-domain threshold, target conditions and forward remainder contract remain the authority for analytic admission.[8]

Only after equality of coefficients and metadata should native measurements compare construction, frontier extension and repeated refinement separately. Small orders may favor the triangular route; a measured crossover can guide method selection. Reusing previously computed core coefficients across observable powers is a further concrete optimization to measure, but must not reuse a result under incompatible source/assumption conditions. This is a specific continuation of P08, not a claim to have already repaired all refinement or provenance issues.

10 P2: a stable numerical route for the real Lambert branch W_{-1}

10.1 Scope of the proposal

The Mathics notes already acknowledge that numerical nonprincipal Lambert evaluation is unsupported. Repeating that statement as a newly discovered bug would not satisfy an incremental review. The additional contribution here is a small, bounded numerical algorithm with a real-branch proof, explicit conditioning analysis, an implemented Python prototype and test values chosen to stress both the branch point and the far tail.[11, 24]

The proposal does not alter `System'ProductLog` and does not claim arbitrary complex-branch support. It targets only $W_{-1}(z)$ for real $-e^{-1} \leq z < 0$. The standard branch relation and branch point are described in DLMF; the logarithmic coordinate and bracket below are derived directly.[40]

10.2 Avoid representing the tiny argument at all

Define

$$\delta = -\log(-z) - 1 \geq 0, \quad z = -e^{-1-\delta}, \quad W_{-1}(z) = -1 - v,$$

where $v \geq 0$. The equation $we^w = z$ reduces to

$$h(v) := v - \log(1 + v) = \delta. \quad (4)$$

This transformation makes both extremes manageable. For very negative logarithmic arguments, one need not construct a tiny z before finding w . Near $z = -e^{-1}$, one can retain the small correction v separately rather than subtracting two nearly equal values after evaluating a branch function.

An API accepting *log excess* δ is useful here. An API accepting an already rounded z very close to the branch point cannot recover a lost sign of δ or missing digits. The prototype therefore accepts exact decimal strings for δ and refuses ordinary Python binary floats. It does not manufacture information absent from an inexact input.

10.3 A global real bracket

Proposition 10.1. *For every $\delta > 0$, equation (4) has a unique positive solution, and*

$$\sqrt{2\delta} \leq v \leq \delta + \sqrt{\delta^2 + 2\delta}. \quad (5)$$

For $\delta = 0$, the solution is $v = 0$.

Proof. For $v > 0$, $h'(v) = v/(1+v) > 0$, $h(0) = 0$, and $h(v) \rightarrow \infty$. Also

$$h(v) = \int_0^v \frac{s}{1+s} ds.$$

On $0 \leq s \leq v$, $s/(1+v) \leq s/(1+s) \leq s$, giving

$$\frac{v^2}{2(1+v)} \leq h(v) \leq \frac{v^2}{2}.$$

Set $h(v) = \delta$. The upper bound on h gives the lower bound in (5); solving $v^2 \leq 2\delta(1+v)$ gives its upper bound. $\square$

The bracket is elementary and does not depend on an unverified branch guess. It also explains the numerical conditioning. Since $h'(v) \rightarrow 0$ as $v \rightarrow 0$, a fixed small equation residual is not a uniform forward-error guarantee near the branch point. Bracket width, measured relative to the small correction, is a more appropriate stopping target.

10.4 Stable phase evaluation and bounded bisection

Direct subtraction in $v - \log(1 + v)$ loses relative accuracy for tiny v . For $0 \leq v \leq 1/2$ the prototype instead sums

$$h(v) = \sum_{n=2}^{\infty} \frac{(-1)^n v^n}{n},$$

with a bounded termination rule. For larger v , it uses $v - \log 1p(v)$. Bisection of (5) then preserves the real branch in exact arithmetic. The implementation provides a finite iteration budget, finite precision caps, validation of the argument and requested digits, and an explicit exhausted-budget exception.

The returned object contains $-1 - v$, v , both correction endpoints, iteration count, working precision and equation residual. It also contains `certified=False`. This last field is deliberate: the proof establishes a mathematical bracket, but the implementation uses ordinary mpmath rounding. Its computed endpoints are not outward-rounded interval enclosures. A future rigorous variant would need directed bounds for square root and the phase evaluation.

The implementation increases working precision near the branch point and returns v separately even then. That prevents the API from reducing a nonzero correction to an unexplained printed -1 . For extremely demanding arguments it refuses the prototype's precision or iteration budget rather than silently weakening the requested correction accuracy.

10.5 Numerical evidence

The test suite requests 70-digit relative accuracy in v and compares with a 200-digit mpmath Lambert oracle at

$$\delta \in \{0, 10^{-60}, 10^{-12}, 0.1, 1, 10, 1000, 10^6\}.$$

All tests passed, including branch-point handling, invalid inputs and deliberate budget exhaustion. For the nonzero cases the observed relative correction errors were below $3 \cdot 10^{-71}$. Selected values are

δ	$W_{-1}(-e^{-1-\delta})$ (rounded)	Bisections
10^{-60}	$-1 - 1.414213562373095 \cdot 10^{-30}$	133
0.1	-1.5162211614250221353	231
1	-3.1461932206205825852	233
1000	-1007.9156397544092181	238
10^6	-1000014.8155253733799	243

The zero case returns -1 exactly in the prototype; a tiny nonzero difference in a finite-precision Lambert reference at the branch point is reference rounding, not a correction claimed by this algorithm. The recorded numerical residuals and endpoint widths remain diagnostics, not certificates.

10.6 A narrow path into Mathics

A Mathics implementation can use ordinary high-precision arithmetic, logarithms, finite integer loops and the bounded bisection, without invoking the defective or unsupported two-argument nonprincipal evaluator. It should be attached to a checked numerical specialization path or an explicit helper, not smuggled into symbolic branch construction. A symbolic `ProductLog[-1, z]` remains an exact symbolic object; converting it numerically is a separate operation.

Acceptance should compare both w and the correction v , include exact branch-point input, invalid real-side input, very small δ , and large δ without materializing z . It should test insufficient supplied precision and show the chosen numerical status. The shipped Python implementation is a working reference, not a native Wolfram/Mathics patch. That boundary avoids repeating the mistake of treating a successful off-kernel prototype as accepted interpreter compatibility.

11 Additional development consequences of the Mathics architecture

11.1 Treat late definition rewrites as versioned transformations

Several adapters operate by replacing selected symbols or expression shapes inside `DownValues`, and one formatting path directly assembles `UpValues`. This is a pragmatic way to preserve the Wolfram implementation while changing Mathics evaluation. It also creates a specific maintenance risk: a harmless refactoring of the original definition can change the shape being matched, leaving a late rewrite with zero matches and no obvious failure. This is an architectural risk inferred from the current mechanism, not a claim that a current rewrite was observed to miss.[24, 19, 17, 20, 21]

A focused improvement is to specify a transformation contract for each late shim: which consumer symbols are transformed, which source patterns are expected, what residual forms are forbidden afterward, and whether reloading is idempotent. Tests should deliberately perturb a miniature definition shape and require either a successful equivalent transformation or a clear compatibility-installation failure. Counting matches can help, but a count alone is not semantic proof; changing a source definition may legitimately require changing the declared contract.

This is narrower than proposing another general capability registry. It addresses the actual `DownValues` transformation boundary introduced by the Mathics port. It also suggests keeping a small diagnostic record of applied transformations for a failed compatibility run, without exposing private implementation symbols as a new stable public API.

11.2 Measure startup separately from mathematical work

Because the portable suite starts a fresh interpreter for every case, load-time rewriting, parsing and initialization are paid repeatedly. Optimizing a symbolic recurrence and optimizing adapter installation affect different parts of the recorded elapsed time. The existing receipts are sufficient to show that startup is non-negligible, but not to attribute a precise percentage of each calculation to it.

A focused benchmark should report three intervals: process startup to first input, package load to readiness, and the selected mathematical operation. Cold-process regression isolation should remain unchanged. A separate benchmark can reuse a loaded process to measure arithmetic without confusing that experiment with cross-case regression isolation. This follows directly from the current test architecture; it is not a proposal to weaken the fresh-kernel tests for speed.

For `FirstPosition`, additionally count visited candidates and materialized matches; for `Gamma`, count phase evaluations and coefficient work; for late shims, count transformed definitions and reload cost. These measurements give more actionable explanations than one aggregate wall-clock number.

11.3 Keep internal proof allowances distinct from user deadlines

The Mathics time adapter multiplies internal proof allowances by four, while selected consumers restore the original handling of explicit core-check deadlines and the callable-application guard. This is already implemented and documented. Future edits should preserve that distinction: an interpreter-overhead allowance for an internal proof attempt is not permission to multiply an explicit user deadline silently.[25, 24]

A concrete regression should install a deliberately slow controlled callback and distinguish an explicitly supplied constraint from an internal default allowance. Such a test should verify outcome and deadline

class, with a tolerant timing window, rather than asserting exact scheduler milliseconds. This is an acceptance refinement for the existing adapter, not a claim that the current explicit-deadline path is broken.

12 Incremental roadmap and novelty ledger

12.1 Recommended work order

The immediate low-risk work is the portable runner patch: it changes neither coefficient mathematics nor kernel definitions and has isolated reproduced witnesses. Before merging, extend its interruption test to Windows and its write-ahead state test to forced termination. Verify that existing report consumers tolerate the new fields and preserve the distinction between attempts and completed results.

Next, close the small adapter characterization gap for `FirstPosition`, keeping a measured pure-result control and an effectful-predicate control. Resolve `Limit`'s default only within its intended private interface; current explicit directions should not be disturbed by a broad rewrite.

Then port the exact Gamma prototype as an alternative internal coefficient provider, with a conservative small-order fallback until measurements justify switching. Compare coefficients, nonzero-block/frontier semantics and final family metadata independently. The broad idea of optimizing Gamma was already recorded; the equation, exact ring and tests here make a bounded implementation task possible.

For numerical Mathics support, the real W_{-1} log-excess helper is a tractable next component. It should expose accuracy and failure information honestly and remain separate from symbolic construction and pointwise certification. Only after native numerical acceptance should it be used to specialize retained negative-branch Lambert cores.

Finally, attach transformation-contract tests to the late Mathics shims when their upstream consumer definitions change. That protects a fragile integration boundary without demanding a wholesale architectural rewrite.

12.2 What was deliberately not re-reported

This audit does not reissue the known native dense-allocation/index issues, assumption capture, generic Taylor-regularity admission, composition scope, refinement margins, target-chart errors, certificate accuracy work, flat-sector tail ordering, backend defaults, option canonicalization, callable routing, native eager `Normal`, symbolic logarithm branch erasure, or the existing general extension wishlist. Their status and evidence remain in the consolidated register and wave-3 intake.[4, 5]

The two mathematical proposals also do not pretend to invent the broader goals of special-function optimization or numerical compatibility. They are included because they add concrete algorithms, proofs, implementations and scoped test evidence to already recognized needs. Where a prior generic item is the nearest predecessor, the ledger says so.

ID	Finding or proposal	Increment beyond earlier work
N1	Portable timeout admission	Actual NaN, infinity and huge-finite subprocess witnesses in the new runner; validated bounded refusal. Different from legacy V02.

ID	Finding or proposal	Increment beyond earlier work
N2	Loader dependency coverage	Official <code>init.m</code> route omits loaded siblings from provenance; mutation witness, candidate coverage repair and explicit root policy.
N3	In-flight interruption record	Real SIGINT witness loses case identity in the original receipt; write-ahead active state and explicit interrupted outcome. Not a false-green allegation.
N4	<code>FirstPosition</code> all-match search	Specific Mathics helper adds avoidable traversal/materialization and an effectful-predicate distinction; new characterization, not an executed public failure.
D1	<code>Limit</code> default mismatch	Native Wolfram 15 controls sharpen the direction discussion. Inspected consumers specify direction, so severity is latent/low.
P1	Gamma formal Newton	Sharpens P08 with a normalized unit-Jacobian equation, exact implementation, independent low-order check and measured prototype work counts.
P2	Real W_{-1} specialization	Goes beyond the documented numerical gap with a stable log-excess coordinate, proved bracket, bounded implementation and stress tests.

12.3 Acceptance matrix for the supplied artifacts

Artifact	Established here	Still required for production
Portable runner patch	Six Python tests with byte-identical original fixture and fake kernels; finite controls, timeout errors, dependency mutations, POSIX interruption.	Windows interruption, forced termination checkpoint, report-consumer compatibility and real-kernel focused acceptance.
Gamma coefficient prototype	Exact triangular/Newton equality through order 8, residual identities, separate symbolic order-4 check, original-function numerical samples.	Native port, complete supported parameter/observable comparisons, retained remainder metadata and matched performance measurements.
W_{-1} prototype	Real bracket proof; 70-digit correction checks on eight log-excess inputs; invalid-input and budget checks.	Mathics implementation and precision semantics; directed rounding only if certification is claimed.
Adapter characterization	Source-identified <code>FirstPosition</code> mechanism and native <code>Limit</code> controls.	Execute supplied private-adapter tests on the supported interpreter; validate any bounded-position form before patching.

Artifact	Established here	Still required for production
CI summaries	Six original receipts preserved; counts and distinct IDs recomputed.	Missing matrix completions remain missing evidence, not inferred successes.

13 Reproduction and bundle contents

13.1 Run the independent checks

Use Python with the two packages in `code/requirements.txt`. The recorded environment was Python 3.13.5, SymPy 1.14.0 and mpmath 1.3.0 on Linux. The code uses modern Python type syntax; Python 3.11 or later is the intended reproduction target. The fake-executable process harness is POSIX-specific and should be run under Linux or WSL; the independent mathematical tests do not require those process features.

```
python -m pip install -r code/requirements.txt
python code/make_harness_candidate.py
python code/test_harness.py
python code/test_mathematical_prototypes.py
python code/summarize_ci.py
```

The first two test files each ran six unittest tests successfully. Subcases and raw observations are retained in JSON. The harness test uses fake executables and temporary source trees; it does not require Mathics or Wolfram. The mathematical tests are independent models, not a hidden execution of the upstream package. Generated test outputs overwrite only this bundle's result files.

The patch generator checks unique text anchors against the included original runner fixture and emits a complete candidate plus a unified diff. It does not apply the diff to a repository or alter the generated standalone Wolfram package. The original fixture's byte identity was checked against the Git blob returned by GitHub; this is recorded by a test, not by a separate checksum manifest.

13.2 Native and Mathics characterizations

For the native smoke reconstruction, point `ASYMPTOTIC_AUDIT_SOURCE` to a local pinned standalone file and execute `code/native_smoke.wls` in a fresh Wolfram kernel. The script performs no network download. The original audit's successful call used `URLDownload` inside the remote kernel; the included file is a readable reconstruction of the observations, not a claim that that exact script file was executed.

For the adapter characterization, load the package in a separate top-level expression before reading `code/adapter_characterization.wl`. The file is marked unexecuted and prints actual versus desired observations. It deliberately tests private adapter behavior and is not a replacement for the package's public regression suite.

The retrieved CI JSON files in `results/ci/` are preserved with their original evidence fields, including embedded source identity metadata. These are test receipts, not added checksum files. Their artifact IDs and run association are listed in the source appendix and metadata. No font files, external-review archives or unrelated repository files are distributed.

13.3 Rebuild the article

The article is self-contained LaTeX with standard packages, New PX text/math fonts, tables and listings. From `article/`, run

```
pdflatex -interaction=nonstopmode -halt-on-error asymptotic-mathics-audit.tex
pdflatex -interaction=nonstopmode -halt-on-error asymptotic-mathics-audit.tex
```

The PDF included in the bundle was compiled from this source and visually checked after rendering. The editable mathematical derivations and source references are retained; no proprietary font files are included.

14 Conclusion

At this revision, AsymptoticAnalysis has a meaningful specialized role beside Wolfram’s built-ins: it makes real weighted scales, selected exact inverse cores, branch information and remainder-aware operations explicit. Native tools retain broader coverage, and the package’s separate native result contract correctly avoids equating that breadth with a custom analytic guarantee.

The Mathics port has moved beyond syntax compatibility into difficult evaluator, assumption, special-function and state-management work. The current artifacts support many successful exact examples, but not completed compatibility acceptance. The new runner findings matter precisely because this distinction should be mechanically visible rather than reconstructed by reading CI logs after cancellation.

The most productive next changes are narrow: repair the three reproduced runner issues; characterize and bound the all-match `FirstPosition` helper; use the exact Gamma prototype to test a concrete coefficient-engine replacement; and implement a checked real W_{-1} numerical path without changing global interpreter semantics. Each has a stated evidence boundary and a finite acceptance task. None requires recycling the already extensive backlog or overstating what the present tests establish.

A Source coverage and preserved artifact provenance

A.1 Repository files inspected

The audit directly inspected the following areas at the pinned revision. A listed module is source coverage, not a claim of complete execution coverage.

Area	Files and responsibilities
Overview and inventory	Root <code>README.md</code> ; <code>src/Kernel/README.md</code> ; repository and source directory trees.
Review exclusion	<code>docs/development/CODE_REVIEW_STATUS.md</code> ; <code>docs/development/WAVE_3_INTAKE.md</code> ; external review index; selected original wave-3 report 23 material.
Core/native boundary	<code>src/Kernel/NativeCompatibility.wl</code> ; selected standalone definitions and call sites imported into the native evaluator; public smoke result.

Area	Files and responsibilities
Special inverse engines	GammaInverse.wl, LambertInverse.wl, FourierCoefficients.wl.
Mathics foundations	MathicsCompatibility.wl, MathicsAssumptions.wl, MathicsSimplification.wl, MathicsCalls.wl, MathicsInverseBranches.wl.
Mathics mathematics	MathicsAlgebra.wl, MathicsTaylor.wl, MathicsSpecialFunctions.wl, MathicsCoreFunctions.wl, MathicsCalculus.wl.
Mathics state and display	MathicsCertificate.wl, MathicsRefinement.wl, MathicsLists.wl, MathicsFormatting.wl, MathicsTimeBudget.wl.
Compatibility evidence	docs/Mathics/COMPATIBILITY.md; .github/workflows/mathics.yml; pinned workflow/job metadata and six complete downloaded artifact files.
Runner/distribution	validation/run_mathics_tests.py; src/Kernel/init.m; root license.

A.2 CI artifact identities

All artifacts belong to run 34428390335 and the pinned commit:

Artifact name	GitHub artifact ID
mathics-modular-foundation	10133621572
mathics-modular-contracts	10133614910
mathics-modular-calculus	10133653224
mathics-modular-special	10133653050
mathics-modular-operations	10133653075
mathics-standalone-foundation	10133653000

The downloaded ZIP containers were used to extract original JSON receipts. The bundle contains those receipts and a reproducible summary, not nested copies of the artifact ZIPs. The workflow status is a retrieval-time observation, not a statement about every later run on a moving main branch.

References

- [1] Vladimir Reshetnikov and contributors. *Asymptotic*, pinned tree and commit. <https://github.com/VladimirReshetnikov/Asymptotic/tree/7d1bc832895cc90a9b2a978b7b7684acab908bd2>.
- [2] Repository *README*. Pinned README.md.
- [3] Repository kernel module guide. `src/Kernel/README.md`.
- [4] Consolidated review register. `CODE_REVIEW_STATUS.md`.
- [5] Wave-3 intake and complete finding crosswalk. `WAVE_3_INTAKE.md`.
- [6] Original report 23, scope and evidence summary. `code-review-23/README.md`.
- [7] Native delegation implementation. `NativeCompatibility.wl`.
- [8] Gamma and Barnes inverse construction. `GammaInverse.wl`.
- [9] Lambert inverse construction. `LambertInverse.wl`.
- [10] Fourier-polynomial coefficient and inverse machinery. `FourierCoefficients.wl`.
- [11] Project Mathics compatibility status and execution guidance. `docs/Mathics/COMPATIBILITY.md`.
- [12] Mathics evaluator primitives and symbol isolation. `MathicsCompatibility.wl`.
- [13] Exact bounded Mathics assumption reasoning. `MathicsAssumptions.wl`.
- [14] Simplification, local series and limit adapters. `MathicsSimplification.wl`.
- [15] Bounded polynomial and affine inverse-branch proofs. `MathicsInverseBranches.wl`.
- [16] Held callable-parameter extraction. `MathicsCalls.wl`.
- [17] Association evaluation in refinement state. `MathicsRefinement.wl`.
- [18] Finite derivative and Euler sums. `MathicsCalculus.wl`.
- [19] Certificate history assignment adapter. `MathicsCertificate.wl`.
- [20] Empty-list map adapter and private installation. `MathicsLists.wl`.
- [21] Arithmetic upvalues and formatting installation. `MathicsFormatting.wl`.
- [22] Bounded defining Taylor-series bridge. `MathicsTaylor.wl`.
- [23] Finite positive-argument Stirling adapter. `MathicsSpecialFunctions.wl`.
- [24] ProductLog normalization and explicit time-constraint preservation. `MathicsCoreFunctions.wl`.
- [25] Internal proof-budget multiplier. `MathicsTimeBudget.wl`.
- [26] Portable fresh-process test runner. `validation/run_mathics_tests.py`.
- [27] Official modular loader. `src/Kernel/init.m`.
- [28] Mathics CI matrix. `.github/workflows/mathics.yml`.
- [29] GitHub Actions run 34428390335 and its job/artifact metadata. <https://github.com/VladimirReshetnikov/Asymptotic/actions/runs/34428390335>.
- [30] Wolfram Research. *Series*. <https://reference.wolfram.com/language/ref/Series.html>.
- [31] Wolfram Research. *SeriesData*. <https://reference.wolfram.com/language/ref/SeriesData.html>.
- [32] Wolfram Research. *InverseSeries*. <https://reference.wolfram.com/language/ref/InverseSeries.html>.
- [33] Wolfram Research. *Asymptotic*. <https://reference.wolfram.com/language/ref/Asymptotic.html>.
- [34] Wolfram Research. *AsymptoticSolve*. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>.
- [35] Wolfram Research. *DiscreteAsymptotic*. <https://reference.wolfram.com/language/ref/DiscreteAsymptotic.html>.
- [36] Wolfram Research. *FirstPosition*. <https://reference.wolfram.com/language/ref/FirstPosition.html>.
- [37] Wolfram Research. *Limit*. <https://reference.wolfram.com/language/ref/Limit.html>.
- [38] Mathics3 project. <https://mathics.org/>. Project identity and implementation context; package-specific compatibility is assessed from the pinned sources and receipts above.
- [39] NIST Digital Library of Mathematical Functions, §5.11, *Asymptotic Expansions of the Gamma function*. <https://dlmf.nist.gov/5.11>.
- [40] NIST Digital Library of Mathematical Functions, §4.13, *Lambert W-Function*. <https://dlmf.nist.gov/4.13>.